BOAZ-STUDY

Recrutement Technique

SPÉCIFICATION FONCTIONNELLE

Test Technique – Développeur Backend Java Spring Boot

Projet : TicketFlow

Système de Gestion de Tickets Support

Version	1.1
Durée	3 jours
Contrat visé	CDD pré-emploi → CDI
Contact	recrutement@boaz-study.com

1. Concept & Objectif du Projet

TicketFlow est un mini-système de helpdesk permettant à des utilisateurs de créer des tickets de support, de recevoir des notifications automatiques et de joindre des documents. Le candidat doit concevoir et implémenter l'ensemble de ce système en respectant une architecture microservices propre, sécurisée et résiliente.

Ce test évalue la capacité du candidat à :

- Concevoir et déployer une architecture microservices complète avec Spring Boot et Spring Cloud
- Implémenter une gestion fine des rôles, permissions et accès (RBAC + ABAC) synchronisée avec Keycloak
- Mettre en place la communication synchrone (FeignClient + Resilience4j) et asynchrone (Kafka)
- Assurer l'observabilité, la documentation et la qualité du code

2. Architecture des Microservices

Le système se compose des éléments suivants :

Service / Composant	Port	Responsabilités
api-gateway	8080	Point d'entrée unique – routing, validation JWT, CORS, rate limiting
user-service	8081	Gestion des utilisateurs, rôles et permissions – sync Keycloak
ticket-service	8082	Gestion des tickets, commentaires, statuts – appels Feign + Resilience4j
notification-service	8083	Consommation des événements Kafka – envoi emails/SMS (mock)
document-service	8084	Upload/download de fichiers – stockage MinIO S3
eureka-server	8761	Service Discovery – enregistrement et découverte des microservices
config-server	8888	Configuration centralisée via Git repo
Keycloak	8180	Gestion centralisée : realm, clients, rôles, permissions (scopes), JWT
Kafka	9092	Message broker – communication asynchrone entre services
MinIO	9000/9001	Stockage de fichiers compatible S3 – console sur port 9001
Zipkin	9411	Tracing distribué – visualisation des traces inter-services
Prometheus	9090	Collecte des métriques applicatives – scraping Actuator

3. Gestion des Rôles, Permissions et Accès (RBAC + ABAC)

Cette section est **critique et non négociable**. L'ensemble du système de contrôle d'accès repose sur les permissions (scopes), et non sur les rôles directement. Keycloak est le référentiel central de cette gestion.

3.1 Modèle de sécurité : RBAC + ABAC

Le système adopte un modèle hybride :

- RBAC (Role-Based Access Control) : les permissions sont regroupées en rôles. Un utilisateur se voit attribuer un ou plusieurs rôles.
- ABAC (Attribute-Based Access Control) : la protection des endpoints est assurée uniquement par les permissions (scopes), jamais par les rôles. Cela permet une granularité fine et une évolutivité maximale.

Règle fondamentale : Toute annotation @PreAuthorize sur un endpoint doit utiliser hasAuthority('scope') et non hasRole('ROLE_X').

3.2 Rôles et Permissions prédéfinis

Le candidat doit configurer les rôles et permissions suivants dans Keycloak ET les synchroniser dans la base de données locale :

Rôle	Permissions (scopes) associées
ADMIN	user:create, user:read, user:update, user:delete, user:manage-roles, ticket:create, ticket:read, ticket:update, ticket:delete, ticket:comment, document:upload, document:read, document:download, notification:read, notification:send
AGENT	ticket:create, ticket:read, ticket:update, ticket:comment, document:upload, document:read, document:download, notification:read
USER	ticket:create, ticket:read, ticket:comment, document:upload, document:read, document:download

3.3 Règles d'héritage et d'attribution

Les règles suivantes doivent être implémentées et synchronisées entre la base de données et Keycloak :

1. Un utilisateur peut se voir attribuer un ou plusieurs rôles.
2. Lorsqu'un rôle est attribué à un utilisateur, il hérite automatiquement de TOUTES les permissions (scopes) associées à ce rôle.
3. Si un utilisateur possède plusieurs rôles, il hérite de l'union de toutes les permissions de ces rôles (sans doublons).
4. La révocation d'un rôle entraîne la suppression immédiate des permissions correspondantes (si non couvertes par un autre rôle).
5. Toute modification de rôle (ajout/suppression de permissions) doit être répercutée sur tous les utilisateurs portant ce rôle.

3.4 JWT et propagation des permissions

À la connexion via Keycloak, le JWT émis doit contenir :

- Les rôles de l'utilisateur dans le claim roles ou realm_access.roles
- L'ensemble des permissions (scopes) héritées de ces rôles dans le claim scope ou authorities
- Exemple : un utilisateur avec les rôles AGENT et USER doit avoir dans son JWT tous les scopes de AGENT ∪ USER

Exemple de payload JWT attendu :

```
{
  "sub": "user-uuid",
  "roles": ["AGENT", "USER"],
  "scope": "ticket:create ticket:read ticket:update ticket:comment document:upload
document:read document:download notification:read",
  "authorities": ["ticket:create", "ticket:read", "ticket:update", ...]
}
```

3.5 Synchronisation Keycloak

La synchronisation entre la base de données locale et Keycloak est obligatoire :

- Création d'un rôle → création du rôle ET de ses scopes dans Keycloak
- Attribution d'un rôle à un utilisateur (PUT /api/users/{id}/roles) → mise à jour des rôles dans Keycloak + regénération des scopes dans le token
- Modification des permissions d'un rôle → mise à jour des scopes du rôle dans Keycloak
- La vérification des permissions dans Spring Security doit utiliser les claims du JWT Keycloak
- Utiliser un JwtAuthenticationConverter configuré pour extraire les authorities depuis le claim approprié

4. Spécifications Détaillées par Microservice

4.1 user-service — Gestion des utilisateurs, rôles et permissions

Ce service est le cœur de la gestion des identités et des accès. Il expose les endpoints suivants :

Endpoint	Méthode	Description	Scope requis
/api/users	POST	Créer un nouvel utilisateur	**user:create**
/api/users/{id}	GET	Récupérer un utilisateur par ID	**user:read**
/api/users/{id}/roles	PUT	Attribuer un ou plusieurs rôles à un utilisateur	**user:manage-roles**
/api/roles	POST	Créer un nouveau rôle	**user:manage-roles**
/api/roles	GET	Lister tous les rôles disponibles	**user:read**
/api/roles/{id}/permissions	PUT	Associer des permissions à un rôle	**user:manage-roles**
/api/permissions	GET	Lister toutes les permissions disponibles	**user:read**

Exigences techniques du user-service :

- Création d'un utilisateur → synchronisation immédiate avec Keycloak (création du compte Keycloak)
- Attribution de rôles → mise à jour des rôles ET des scopes dans Keycloak pour l'utilisateur
- Les permissions d'un utilisateur (union de ses rôles) doivent être visibles dans son JWT à la prochaine connexion
- Publication événement Kafka user.created après création
- Stockage en PostgreSQL (tables : users, roles, permissions, user_roles, role_permissions)
- Documentation Swagger complète

4.2 ticket-service — Gestion des tickets

Endpoint	Méthode	Description	Scope requis
/api/tickets	POST	Créer un nouveau ticket	**ticket:create**
/api/tickets	GET	Lister les tickets (paginé)	**ticket:read**
/api/tickets/{id}	GET	Détail d'un ticket	**ticket:read**
/api/tickets/{id}/status	PATCH	Changer le statut (OPEN → IN_PROGRESS → RESOLVED → CLOSED)	**ticket:update**
/api/tickets/{id}/comments	POST	Ajouter un commentaire	**ticket:comment**

Exigences techniques du ticket-service :

- Appel FeignClient vers user-service pour valider l'assignee (avec propagation du JWT)
- Appel FeignClient vers document-service pour lier des pièces jointes
- Resilience4j : CircuitBreaker + Retry + Fallback sur tous les appels Feign
- Publication événements Kafka : ticket.created, ticket.status.changed
- Stockage PostgreSQL

4.3 notification-service — Notifications et consommateurs Kafka

Endpoint	Méthode	Description	Scope requis
/api/notifications/history	GET	Historique des notifications envoyées	**notification:read**
/api/notifications/send	POST	Envoi manuel d'une notification (test)	**notification:send**

Consommateurs Kafka à implémenter :

- user.created → envoi email de bienvenue (log ou mock accepté)
- ticket.created → notification au support
- ticket.status.changed → notification à l'utilisateur concerné

Exigences techniques :

- Spring Kafka avec annotation @KafkaListener (topics + groupId correctement configurés)
- Simulation d'envoi email/SMS via logs ou service mock
- Historique des notifications stocké en base

4.4 document-service — Gestion des fichiers

Endpoint	Méthode	Description	Scope requis
/api/documents/upload	POST	Upload d'un fichier vers MinIO (multipart/form-data)	**document:upload**
/api/documents/{id}	GET	Récupérer les métadonnées d'un document	**document:read**
/api/documents/{id}/download	GET	Obtenir une URL présignée valable 5 minutes	**document:download**

Exigences techniques :

- Intégration MinIO S3-compatible (stockage effectif des fichiers)
- Génération d'URL présignée avec expiration de 5 minutes
- Métadonnées (nom, taille, MIME, date) stockées en PostgreSQL
- Publication événement Kafka document.uploaded après upload

5. Matrice des Événements Kafka

Topic	Producteur	Consommateur(s)	Action déclenchée
user.created	user-service	notification-service	Email de bienvenue
ticket.created	ticket-service	notification-service	Notification au support
ticket.status.changed	ticket-service	notification-service	Notification à l'utilisateur
document.uploaded	document-service	ticket-service	Liaison automatique au ticket

6. Technologies à Implémenter

Technologie	Où l'implémenter	Vérification attendue
Spring Cloud Gateway	api-gateway	Routes /api/**, JWT filter, CORS, rate limiting
Spring Security + Scopes	Tous les services	@PreAuthorize("hasAuthority('scope')") — jamais hasRole()
Keycloak (RBAC+ABAC)	Infrastructure + user-service	Realm ticketflow, rôles, scopes, sync utilisateurs
JWT (Keycloak)	Tous les services	Validation token, extraction scopes → authorities Spring Security
FeignClient	ticket-service	Appels déclaratifs vers user-service et document-service
Resilience4j	ticket-service	CircuitBreaker + Retry + Fallback sur appels Feign
Eureka	Tous les services	Enregistrement automatique + découverte par nom de service
Config Server	Infrastructure	Configuration centralisée via repository Git
Kafka + Spring Kafka	user, ticket, notification, document	@KafkaListener + KafkaTemplate – 4 topics
Sleuth + Zipkin	Tous les services	TraceId propagé dans les logs et visualisable dans Zipkin
Prometheus + Actuator	Tous les services	Métriques exposées via /actuator/prometheus
Springdoc OpenAPI	Tous les services	Swagger UI + agrégation via Gateway
MinIO S3	document-service	Upload, download, URL présignée – bucket ticketflow-bucket

7. Livrables Attendus

Le candidat doit livrer un repository Git (GitHub/GitLab) contenant :

```
ticketflow/
├── docker-compose.yml          # Infrastructure complète (1 commande)
├── eureka-server/
├── config-server/
├── api-gateway/
├── user-service/
├── ticket-service/
├── notification-service/
├── document-service/
├── config-repo/                # Configurations Git
│   ├── application.yml
│   ├── user-service.yml
│   └── ...
└── README.md                   # Documentation + instructions de démarrage
```

8. Critères d'Évaluation

Critère	Points	Vérification
Architecture Gateway	/10	Routes, JWT filter, CORS, rate limiting
Sécurité Keycloak + Scopes (RBAC+ABAC)	/20	Config realm, rôles, permissions, @PreAuthorize(scope), JWT
Gestion Rôles & Permissions (user-service)	/15	CRUD rôles, association permissions, héritage, sync Keycloak
Communication Sync (FeignClient)	/10	Interfaces déclaratives, propagation JWT
Résilience (Resilience4j)	/10	CB + Retry + Fallback fonctionnels
Discovery (Eureka)	/5	Services enregistrés et découverts
Config centralisée	/5	Config Server + Git repo
Communication Async (Kafka)	/15	4 topics – events publiés et consommés
Tracing (Sleuth/Zipkin)	/5	TraceId visible dans logs et Zipkin
Métriques (Prometheus)	/5	Endpoints /actuator/prometheus exposés
Documentation (Swagger)	/5	Swagger UI fonctionnel + agrégation Gateway
Stockage MinIO	/10	Upload, download, URL présignée fonctionnels
Qualité code & Structure	/5	Clean code, structure multi-module, README
TOTAL	**/120**	

9. Planning Suggéré (3 jours)

Jour	Tâches
Jour 1	Infrastructure Docker (Eureka, Config Server, Keycloak, Kafka, MinIO, Zipkin, Prometheus) — Configuration du realm Keycloak (rôles, permissions, scopes) — user-service avec gestion complète des utilisateurs, rôles et permissions + sync Keycloak
Jour 2	ticket-service (FeignClient + Resilience4j + Kafka) — document-service (MinIO + Kafka) — notification-service (consommateurs Kafka)
Jour 3	API Gateway (routing, JWT filter, CORS) — Swagger aggregation — Tracing Sleuth/Zipkin — Tests complets — README et documentation

Bonne chance ! Nous évaluons non seulement le résultat mais aussi la rigueur de votre démarche, la clarté de votre code et la qualité de votre documentation.

Contact : recrutement@boaz-study.com